Workshop 2

Hector Fabian Cabrera Vargas

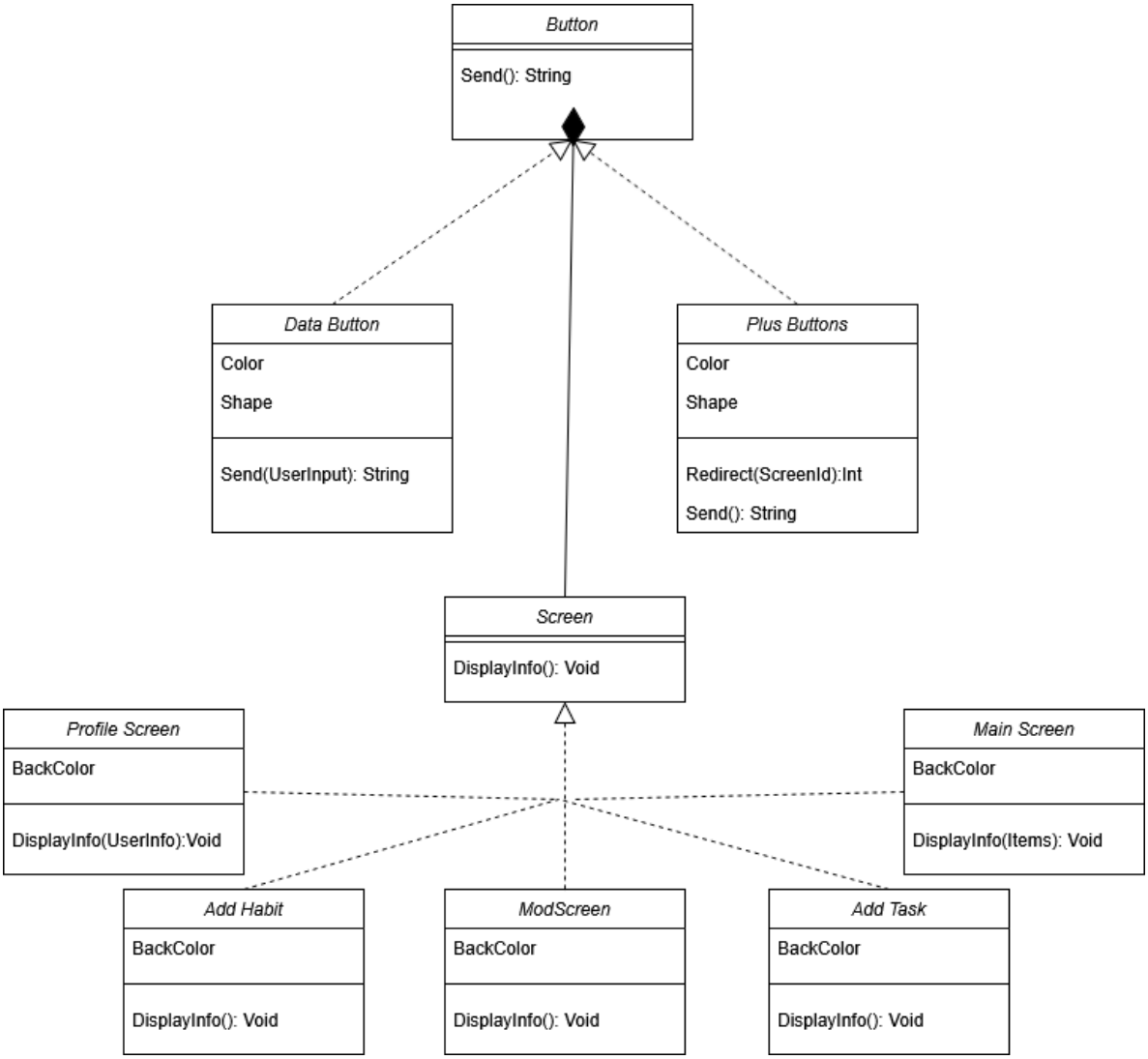
2020020

Juan Esteban Ávila Bautista

2020030

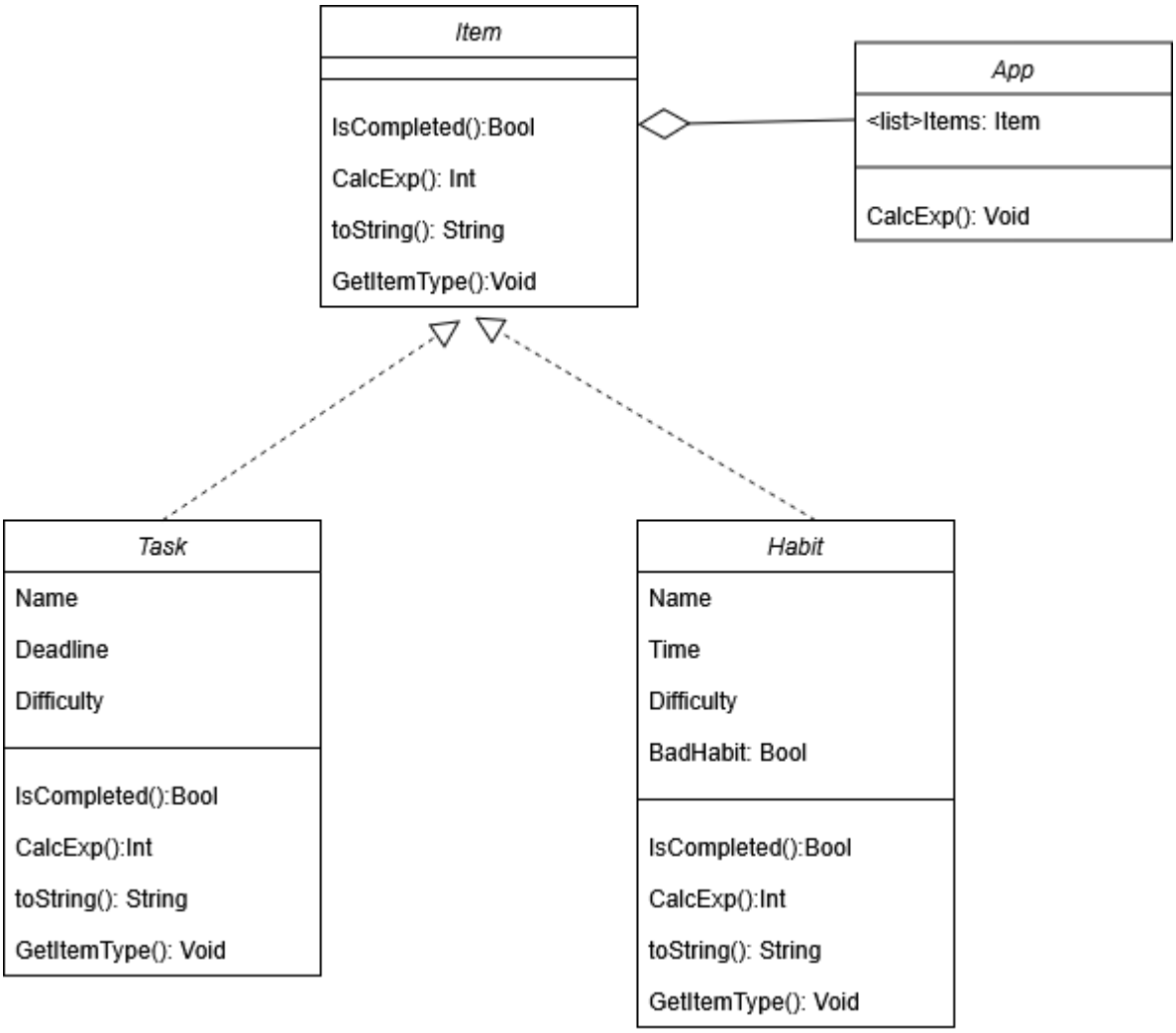
Implementation of SOLID principles.

New UML diagrams:



UML diagram showing Interface use

This diagram shows how different classes make use of an interface, which helps to comply with



New Item classes

This diagram adds a new Item interface, implemented by the Habit and Task classes.

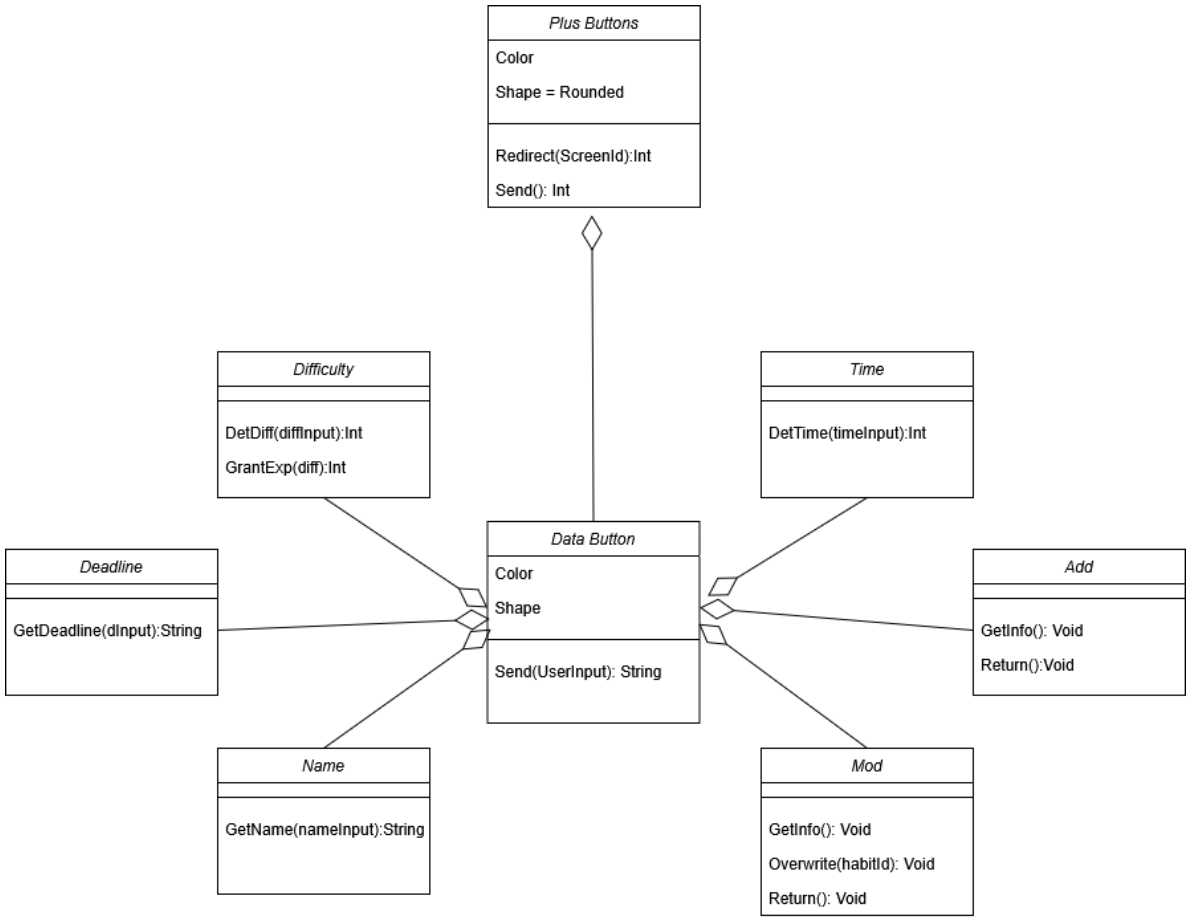


Diagram showing how buttons are implemented

This diagram shows how each class inherits from their respective mother class.

Along with these new class diagrams, a Flowchart was created to highlight how the application works as an algorithm.

SOLID implementation:

1) Single responsibility:

   Single responsibility is achieved when each class and object have a unique role and responsibility that doesn't overlap with one another. In Habitras case, each class is designed in such a way that it complies with this principle, by having multiple classes that assure the flow of information in the application by each processing the information they are concerned about, without displaying any information and vice versa.

2) Open/Closed:

Open/Closed tells us that “classes should be open to extension, but closed to modification”,what this means is that we should be able to add more functionalities without modifying the class itself. Habitra does this by having multiple interfaces that can be easily accessed by other classes to check instances of said interfaces, while also allowing us to add new methods and behaviors without modifying any of the already existing methods in any other class.

3) Liskov Substitution:

   Liskov Substitution is, in simple terms, the ability of a child class to replace its mother or base class. In Habitras case, this principle does not apply, since all the classes have a very concrete and strict hierarchy along with their responsibilities.

4) Interface Segregation:

   Interface Segregation says that no class should be forced to use methods or interfaces it doesn’t need. Habitra complies with this principle by having classes that implement interfaces in a clear and practical manner, using universal and abstract methods that apply to every class that implements said interface.

5) Dependency Inversion:

   Dependency Inversion states that no class should connect to low level modules or concretions, but abstractions instead, this means that classes should depend on interfaces and other high-level classes. Habitra achieves this by connecting its classes via interfaces and not directly with each other.

Work in progress

1)Interfaces:

```java
/**This is an interface for Tasks and habits */

public interface Item{
    public abstract Boolean IsCompleted();
    public abstract Integer CalcExp();
    public abstract String toString();
}
```

This is an interface called item that is used with the classes habit and task.

It has three methods used and useful in every class wich use this interface

```java
//**This is an interface for screens */
public interface Screen {
    public abstract void DisplayInfo();
}
```

This is an interface called Screen that is used to all the screens classes in the app like main screen , add screen , mod screen , etc .

It has only one method which is DisplayInfo wich is used in every class that use this interface.

```java
//**This is an interface for Buttons */
public interface Button {
    public abstract String Send();
}
```

This is an interface called Button that is used to all button classes like add button , mod button , difficulty button , etc.

It has only one method which is Send wich is used in every class that use this interface.

2)Items

Habit

```java
/**This is a class for Habits */
public class Habit implements Item{
    private String habName;
    private int time;
    private int exp;
    private int diff;
    private Boolean status;

    public Boolean IsCompleted(){
        this.status=false;
        return this.status;
    }

    public Integer CalcExp(){
        if(this.diff==1){
            this.exp=1;
        }
        else if(this.diff==2){
            this.exp=2;
        }
        else if(this.diff==3){
            this.exp=3;
        }
        return exp;
    }

    public String toString(){
        return habName+time+diff;
    }
}
```

Task

```java
/**This is a class for tasks */
public class Task implements Item{
    private String taskName;
    private String deadline;
    private int exp;
    private int diff;
    private Boolean status;

    public Boolean IsCompleted(){
        this.status=false;
        return this.status;
    }

    public Integer CalcExp(){
        if(this.diff==1){
            this.exp=1;
        }
        else if(this.diff==2){
            this.exp=2;
        }
        else if(this.diff==3){
            this.exp=3;
        }
        return exp;
    }

    public String toString(){
        return taskName+deadline+diff;
    }
}
```

Task class

3)Screens:

```java
/**
 * This class displays the main menu.
 * It displays all habits.
 */
import java.util.*;
public class MainScreen implements Screen {
    private List<Item> Items = null;
    public void DisplayInfo(){
        if(this.Items.isEmpty()){
            System.out.println("There are no habits or tasks!");
            return;
        }
        for(Item item:this.Items){
            System.out.println(Item.toString());
        }
    }
}
```

Main screen

4)Buttons:

Mod button

```java
import java.util.Scanner;
import mypackage.NameB;

// ModB class extends DataB and provides methods to modify and process a name.
// It allows setting a name, processing received data, and interactively updating the name.
public class ModB extends DataB {
    private String name = null;

    public ModB(String name) {
       this.name = new NameB(name);
    }

    public String Process(String receivedData) {
        return "Modified " + this.name + " and info: " + receivedData;
    }

    /**
     * This method prompts the user to enter a new name for the current object via the console,
     * reads the input, and updates the object's name using the OverWrite method.
     * Displays a message indicating the modification process.
     *
     * @return void
     */
    public void GetInfo() {
        System.out.println("Modifying " + this.name + ". Please enter new name:");
        Scanner scanner = new Scanner(System.in);
        String newName = scanner.nextLine();
        OverWrite(newName);String result = OverWrite(newName);
    }
    /**
     * This method updates the name of the current object with the provided new name.
     * It returns the updated name as a String.
     *
     * @param newName The new name to set for the current object.
     * @return The updated name.
     */
    public String OverWrite(String newName) {
        this.name = newName;
        return this.name ;
    }
```

Difficulty button

```java
import java.util.sacanner;
// This is a class for the buttons that are used to set the difficulty level of a task.
// DifficultyB class extends DataB and provides methods to set and process difficulty levels.
public class DifficultyB extends DataB {
    private String name = null;
    private String difficultyInput = null;

    public DifficultyB(String name) {
        this.name = name;
    }

    public String Process(String receivedData) {
        return "Processed difficulty for " + this.name + " with info: " + receivedData;
    }
    /**
     * This method prompts the user to enter a difficulty level for the current object via the console,
     * reads the input, and returns the difficulty level.
     * If the input is invalid, it defaults to "Easy".
     *
     * @return The difficulty level as a String.
     */
    public String GetDifficulty(difficultyInput) {
        System.out.println("Please enter the difficulty level (Easy, Medium, Hard):");
        Scanner scanner = new Scanner(System.in);
        String difficultyInput = scanner.nextLine();

        if (difficultyInput.equalsIgnoreCase("Easy") ||
            difficultyInput.equalsIgnoreCase("Medium") ||
            difficultyInput.equalsIgnoreCase("Hard")) {
            return difficultyInput;
        } else {
            System.out.println("Invalid input. Defaulting to Easy.");
            return "Easy";
        }
    }
}
```

Deadline button

```java
import java.time.LocalDateTime;
import java.util.scanner;
// This is a class for the buttoms that are used to set the deadline for a task.
// DeadlineB class extends DataB and provides methods to set and process deadlines.
public class DeadlineB extends DataB {
    private String name = null;
    private String timeInput = null;

    public TimeB(String name) {
        this.name = name;
    }

    public String Process(String receivedData) {
        return "Processed time for " + this.name + " with info: " + receivedData;
    }
    /**
     * This method prompts the user to enter a deadline in the format 'yyyy-mm-dd ',
     * reads the input, and returns the deadline as an integer representing seconds since epoch.
     *
     * @param timeInput The input string for the deadline.
     * @return The deadline in seconds since epoch.
     */
    public Integer DetTime(String timeInput) {

        System.out.println("Please enter the deadline in the format 'yyyy-mm-dd ':");
        Scanner scanner = new Scanner(System.in);
        timeInput = scanner.nextLine();

        java.time.format.DateTimeFormatter formatter = java.time.format.DateTimeFormatter.ofPattern("yyyy-mm-dd ");
        LocalDateTime deadline = LocalDateTime.parse(timeInput, formatter);
        return (int) deadline.atZone(java.time.ZoneId.systemDefault()).toEpochSecond();

        }

    }
}
```

Changes to previous design

1) Added Item classes

   These are the core and main way of the app to work, and share behaviors, which allowed us to consider them as classes that share from the same interface.

2) Removed Color button classes

   While they do have different behaviors, they all are fundamentally the same; their purpose is to redirect the user to another screen based on its

id. Upon consideration, they were removed, and instead will be initiated as three different objects, with different attributes.

No further changes were made to the previous design choices; requirements and user stories remain unchanged.